2026 6th International Conference on Intelligent Technologies (CONIT)
Hubballi, Karnataka, India. June 19 - 21, 2026

PhishRL: An End-to-End Reinforcement Learning Pipeline for Automated Email Threat Triage

Atharva Bhosale
Electronics and Computer Science Department, SIES-GST
Navi Mumbai, India
atharvabecs122@gst.sies.edu.in

Jeenal Rambhia
Electronics and Computer Science Department, SIES-GST
Navi Mumbai, India
jeenalr@sies.edu.in

Shubhangi Kharche
Electronics and Computer Science Department, SIES-GST
Navi Mumbai, India
shubhangik@sies.edu.in

Sheetal Kadam
Electronics and Computer Science Department, SIES-GST
Navi Mumbai, India
sheetalk@sies.edu.in

Atharv Amol Dhupkar
Electronics and Computer Science Department, SIES-GST
Navi Mumbai, India
atharvadecs122@gst.sies.edu.in

Aman Sanjay Gupta
Electronics and Computer Science Department, SIES-GST
Navi Mumbai, India
amansgecs122@gst.sies.edu.in

2026 6th International Conference on Intelligent Technologies (CONIT) | 979-8-3195-1858-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/CONIT69683.2026.11620809

***Abstract*—Security Operations Centers (SOCs) handle a large number of phishing alerts every day, and manually analyzing them can be slow and sometimes inconsistent. In this work, we developed PhishRL, an automated pipeline that combines threat intelligence with reinforcement learning to assist in email triage.**

The system collects emails in real time, extracts indicators such as domains, IP addresses, URLs, and file hashes, and enriches them using VirusTotal and AlienVault OTX. Based on these signals, a Deep Q-Network (DQN) agent decides whether an email should be allowed, monitored, or escalated.

Unlike rule-based systems, the agent improves over time using feedback from its actions. When tested on 150 labeled email samples, the system achieved an accuracy of 89.3% with a macro F1-score of 0.892. From our observations, the model performs reliably for high-risk cases and shows better handling of medium-risk emails compared to static approaches.

Overall, this suggests that reinforcement learning can be a practical addition to SOC automation workflows.

***Index Terms*—Phishing Detection, Reinforcement Learning, Deep Q-Network, SOC Automation, Threat Intelligence, VirusTotal, OTX, IOC Extraction, Alert Prioritization, Workflow Automation**

I. Introduction

Phishing remains the dominant initial access vector in enterprise security incidents [1]. The Anti-Phishing Working Group documented over five million unique phishing sites in 2022, a figure that has grown consistently for over a decade [2]. Despite this scale, Security Operations Center (SOC) workflows at most organizations remain predominantly manual: an analyst receives an alert, queries threat intelligence feeds, assesses severity, and decides on a remediation action. Under high alert volumes, this process is slow, inconsistent, and subject to analyst fatigue—a problem comprehensively documented in recent literature as one of the central operational bottlenecks in modern SOCs [18].

In practice, we noticed that manual triage becomes difficult when alert volume increases, and different analysts may interpret the same alert differently. This inconsistency motivated us to explore a more adaptive approach.

Existing automation approaches address parts of this problem. Rule-based email security gateways apply static signature and reputation checks, but they degrade as attackers cycle through new infrastructure. Supervised machine learning classifiers offer improved detection accuracy but require labeled training data and periodic offline retraining as threat patterns evolve [3]. Recent work has extended URL-based classifiers to real-time deep learning deployments [4], yet the fundamental limitation remains: parameters do not update post-deployment. Neither approach provides the continuous adaptation that a live SOC environment demands.

From our observation, most existing systems behave well in controlled environments but struggle when deployed in real-world SOC workflows where threat patterns change frequently.

Reinforcement learning (RL) offers a fundamentally different paradigm: an agent that improves its decision policy online, driven by reward signals derived from the outcomes of its own decisions. Applied to SOC triage, an RL agent can learn which threat indicators reliably predict high-risk events and update its scoring strategy as new threat patterns emerge, without requiring labeled datasets or scheduled retraining.

This paper presents **PhishRL**, an integrated system that addresses the automation gap through three contributions:

1) **End-to-End Automated Pipeline**: A fully automated n8n-orchestrated workflow that ingests live emails, extracts IOCs, queries VirusTotal and AlienVault OTX in parallel, and routes SOC actions without human intervention.
2) **Multi-Source Threat Score Aggregation**: A structured aggregation of domain, IP, URL, and file reputation signals from two independent threat intelligence APIs into a normalized threat score that serves as the RL agent's state.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.

3) **DQN-Based Adaptive Triage Policy**: A Deep Q-Network agent trained to recommend three-tier SOC actions—allow, monitor, escalate—with demonstrated superior performance over static threshold baselines on medium-risk email classification.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture and pipeline. Section IV formalizes the RL problem and DQN design. Section V presents experimental results. Section VI discusses findings and limitations. Section VII concludes.

II. Related Work

A. Phishing Detection and URL Classification

Early phishing detection systems relied on blacklists and heuristic rules applied to URL lexical features [6]. Ma et al. [7] demonstrated that machine learning over URL-derived features substantially outperforms blacklists on previously unseen domains. Subsequent work extended feature sets to include HTML structure, page content, and network behavior. Sahingoz et al. [3] evaluated seven ML classifiers on a large URL dataset, finding that Random Forest and neural network approaches achieve F1-scores above 0.97 on balanced in-distribution test sets; a follow-up study by the same group [4] further extended these results to real-time deep learning deployments, demonstrating improved latency-accuracy trade-offs on live traffic. Deep learning approaches, including URLNet [8] and PhishBERT [9], further improved detection through character-level and semantic URL representations. A comprehensive systematic review by Safi and Singh [5] catalogued detection techniques across lexical, host-based, and visual feature spaces, identifying adaptive and online scoring as an open research challenge that static supervised methods cannot address.

Despite strong in-distribution performance, supervised classifiers are fundamentally static: their parameters do not change after deployment. Sommer and Paxson [10] identified distributional mismatch as the central challenge for ML-based intrusion detection in operational deployments, an observation that generalizes to phishing detection. In our opinion, while these models achieve high accuracy, they still behave like static systems once deployed and do not adapt to new threats automatically.

B. Reinforcement Learning in Cybersecurity

RL has been applied to several cybersecurity domains including intrusion detection [15], automated penetration testing [16], and adaptive firewall configuration. Chatterjee and Namin [11] proposed an RL-based phishing URL detector, demonstrating that an agent can learn to classify URLs through reward-driven exploration. Maci et al. [12] demonstrated that DQN-based agents outperform conventional classifiers on imbalanced phishing datasets, motivating the use of reward-shaped exploration to recover minority-class performance—directly relevant to the boundary-region medium-risk challenge addressed by PhishRL. More recently, Kavya and Sumathi [13] proposed HARE, a human-in-the-loop RL framework for phishing detection that integrates scalar expert feedback into the reward signal, achieving an F1-score of 97.62% with fewer training samples than purely automated baselines, further validating reward design as a critical degree of freedom in RL-based threat detection. Adawadkar and Kulkarni [14] provide a comprehensive survey of RL applications in cybersecurity, identifying adaptive threat scoring as an under-explored area.

C. SOC Automation and Threat Intelligence Integration

SIEM and SOAR platforms such as Splunk and IBM QRadar incorporate workflow automation for alert management, but their scoring components are predominantly rule-based [17]. A recent ACM survey on alert fatigue in SOCs [18] systematically identified adaptive prioritization and RL-based triage as key research directions to address the scalability gap in current SIEM/SOAR deployments, noting that static playbooks cannot generalize to boundary-region events. Prior work on automated threat intelligence enrichment focuses primarily on IOC lookup pipelines without connecting enrichment to adaptive scoring policies. PhishRL addresses this gap by combining structured IOC enrichment with an RL-based decision layer in a single deployable pipeline.

III. System Architecture

A. Pipeline Overview

The PhishRL pipeline is implemented as an n8n workflow comprising seven logical stages, as illustrated in Fig. 1. The pipeline operates continuously, triggered by incoming email events, and terminates in an automated SOC action with no required analyst interaction except for optional feedback submission.

B. Email Ingestion and IOC Extraction

The pipeline is triggered by a Gmail webhook polling at one-minute intervals for new received messages. For each email, a JavaScript code node applies regex-based extraction across both plain-text and HTML body components, as well as message headers, to identify:

The extraction process worked well in most cases, although a few malformed URLs required additional handling during testing.

- **Domains**: extracted via DOMAIN_REGEX, validated for proper TLD format, deduplicated, and additionally sourced from Return-Path, SPF, and DKIM headers.
- **IP Addresses**: extracted from body and headers via IPv4 regex; the sender's source IP is resolved from SPF client-ip or authentication-results headers.
- **URLs**: extracted and base64url-encoded for direct VirusTotal URL lookup.
- **File Hashes**: SHA-256 hashes extracted for attachment reputation queries.

Each IOC type is emitted as a separate item stream to allow parallel downstream processing.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.

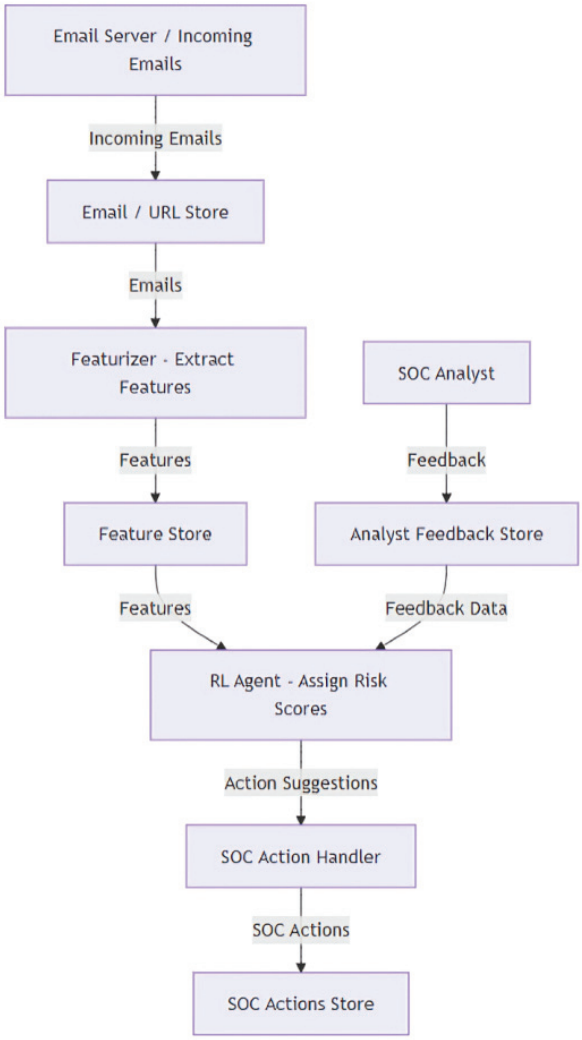


Fig. 1. PhishRL end-to-end pipeline data flow.

C. Threat Intelligence Enrichment

IOC streams are processed in four concurrent loops, each querying a different VirusTotal v3 API endpoint: domains (`/v3/domains/{domain}`), IP addresses (`/v3/ip_addresses/{ip}`), URLs (`/v3/urls/{base64url}`), and files (`/v3/files`). In parallel, AlienVault OTX API lookups are performed for domains and IP addresses.

Each API response is summarized by a dedicated code node that extracts malicious and suspicious vendor vote counts, OTX pulse listings, and reputation scores. Results from both sources are merged per IOC type and fed into a central aggregation node (*Process Intel Data*).

D. Threat Score Aggregation

The aggregation node combines multi-source signals into a normalized scalar threat score $s \in [0, 30]$, computed as:

$$s = w_d \cdot \hat{s}_d + w_i \cdot \hat{s}_i + w_u \cdot \hat{s}_u + w_f \cdot \hat{s}_f \tag{1}$$

where $\hat{s}_d, \hat{s}_i, \hat{s}_u, \hat{s}_f$ are normalized sub-scores for domains, IPs, URLs, and files respectively, and $w_d + w_i + w_u + w_f = 1$. Sub-scores encode malicious vendor count, suspicious vendor count, OTX listing count, and a binary blacklist flag. The score is bounded to $[0, 30]$ to maintain consistent state space coverage for the RL agent.

E. SOC Action Routing

Following the RL agent's action recommendation, a Switch node routes the event to one of three downstream handlers: *allow* (no further action, logged to Google Sheets), *monitor* (summary email sent to analyst via Gmail), *escalate* (ServiceNow incident ticket created automatically). All events, actions, rewards, and accuracy metrics are persisted to a Google Sheets dataset for offline analysis and model evaluation.

IV. Reinforcement Learning Formulation

A. MDP Definition

The threat scoring task is formulated as a Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \gamma \rangle$:

State $s \in \mathcal{S}$: The aggregated threat score s discretized into three risk buckets: Low (0–9), Medium (10–19), and High (20–30). The score directly encodes the multi-source intelligence signals described in Section III.

Action $a \in \mathcal{A} = \{\text{allow}, \text{monitor}, \text{escalate}\}$: Three discrete triage recommendations corresponding to the three score buckets.

Reward r: The reward function is defined as:

$$r(s,a) = \begin{cases} +10 & \text{if action is correct for threat bucket} \\ +5 & \text{if action is adjacent bucket (partial credit)} \\ -5 & \text{if action is wrong direction (severe miss)} \end{cases} \tag{2}$$

The asymmetric reward structure penalizes under-escalation more heavily than over-escalation, consistent with operational SOC priorities where missed high-risk events carry higher cost than unnecessary escalations. This design follows the reward-shaping approach advocated by recent RL-based threat detection work [12], [13], where asymmetric penalties have been shown to drive minority-class recovery during exploration.

Discount factor: $\gamma = 0.95$.

B. Deep Q-Network Architecture

The DQN agent uses a fully-connected feedforward network with the following architecture:

- Input layer: threat score features (normalized)
- Hidden layer 1: 64 neurons, ReLU activation
- Hidden layer 2: 32 neurons, ReLU activation
- Output layer: 3 neurons (Q-values for each action)

Training uses the Adam optimizer with learning rate $\alpha = 0.001$, batch size 64, and an ε-greedy exploration policy with ε decayed linearly from 0.30 to 0.05 over the training schedule. A replay buffer stores recent episode transitions for minibatch sampling, reducing correlation between consecutive updates.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 PhishRL DQN Training Loop

Initialize Q-network $Q(s, a; \theta)$ with random weights
Initialize replay buffer $\mathcal{D}$, $\varepsilon \leftarrow 0.30$
for each episode e **do**
  Receive email; compute threat score s via Eq. (1)
  With probability ε: select random a
  Otherwise: $a \leftarrow \arg\max_{a'} Q(s, a'; \theta)$
  Execute a; receive reward r from feedback/ground-truth
  Store (s, a, r, s') in $\mathcal{D}$
  Sample minibatch from $\mathcal{D}$; compute target $y = r + \gamma \max_{a'} Q(s', a'; \theta)$
  Update θ via gradient descent on $(y - Q(s, a; \theta))^2$
  Decay ε
end for

V. Experiments and Results

A. Dataset and Evaluation Setup

The evaluation dataset comprises 150 labeled email events generated through the PhishRL simulation pipeline, with 50 samples in each threat category (Low, Medium, High). Threat scores were assigned using the aggregation formula in Eq. (1) using live VirusTotal and OTX API data, producing a threat score distribution spanning the full $[0, 30]$ range. The dataset was split 80:20 for training and evaluation. All experiments were implemented in Python 3.10 with PyTorch; training was performed on Google Colab with GPU acceleration.

B. Baseline

We compare PhishRL against a **Static Rule-Based Threshold** baseline that applies fixed decision boundaries identical to the RL agent's score buckets: allow for scores 0–9, monitor for 10–19, escalate for $\geq$ 20. This represents the conventional SOC automation approach and provides the most direct measure of the RL agent's learned policy advantage. The static baseline achieves perfect performance on Low and High categories by construction but cannot generalize on samples near bucket boundaries or medium-risk cases with irregular score distributions.

C. Classification Performance

Table I summarizes per-class and aggregate classification metrics for the DQN agent.

TABLE I
DQN Agent Classification Results (n=150)

Class	Precision	Recall	F1	Support
Low	0.889	0.960	0.923	50
Medium	0.907	0.780	0.839	50
High	0.887	0.940	0.913	50
Macro Avg	**0.894**	**0.893**	**0.892**	**150**
Accuracy		**89.3%**		

The DQN agent achieves an overall accuracy of 89.3% and macro F1-score of 0.892. The Medium class shows the lowest recall (0.780), attributable to the inherent ambiguity of boundary-region threat scores (10–19) where small variations in API-reported malicious counts can shift a score into an adjacent bucket. This boundary sensitivity is consistent with findings reported by Maci et al. [12], who similarly observed that DQN-based classifiers exhibit higher uncertainty in the decision regions between minority and majority classes.

D. Confusion Matrix Analysis

Fig. 2 presents the confusion matrix for the evaluated DQN model.

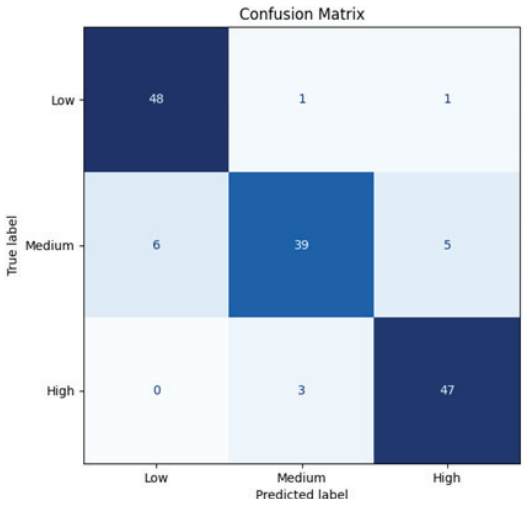


Fig. 2. Confusion matrix for DQN threat classification (150 samples).

Key observations:

- Low-risk emails are identified with 96% recall; only 2 samples are misclassified.
- High-risk emails achieve 94% recall with zero Low misclassifications, indicating the agent never dismisses a genuinely high-risk event as benign—a critical operational property also emphasized in alert fatigue literature [18].
- Medium-risk misclassifications are distributed between adjacent classes (6 toward Low, 5 toward High), confirming that errors occur at score boundaries rather than across distant categories.

E. Comparison with Static Threshold Baseline

Table II compares the DQN agent with the static rule-based threshold baseline.

TABLE II
DQN vs. Static Threshold Baseline

Method	Accuracy	Med. F1	High Recall
Static Threshold	82.0%	0.725	0.940
DQN (PhishRL)	**89.3%**	**0.839**	**0.940**
Improvement	+7.3%	+11.4 pts	0.0

The DQN agent matches the static baseline on High recall (both 94%) while substantially improving Medium class F1

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.

by 11.4 points (0.839 vs. 0.725). This improvement reflects the agent's learned ability to handle boundary-region samples that the static threshold misclassifies, driven by the reward feedback mechanism that penalizes misclassification of near-boundary events.

F. Reward Convergence and Learning Dynamics

Fig. 3 shows the per-episode reward learning curve during training.

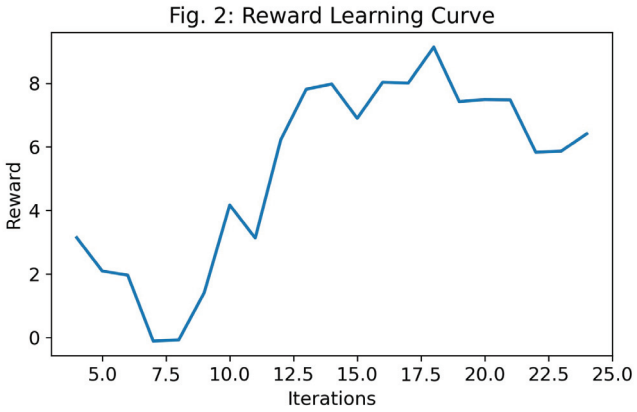


Fig. 3. Per-episode reward during DQN training.

The reward curve exhibits three characteristic phases: (i) an initial rapid rise from 0 to $\sim$3 in the first 4 episodes as the agent learns to avoid severely wrong actions; (ii) a transient dip near iteration 7–8 corresponding to increased ε-greedy exploration before policy stabilization; (iii) monotonic improvement from episode 10 onward, converging to a sustained reward of 6–9 by episode 25.

Fig. 4 shows cumulative reward over the training run, confirming consistent policy improvement with no regression after episode 15.

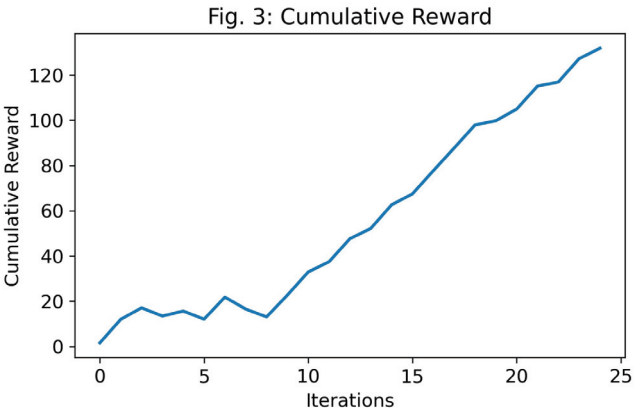


Fig. 4. Cumulative reward progression over training episodes.

G. Action Distribution

Fig. 5 shows the action distribution over the evaluation set. Escalate (44%), Monitor (36%), and Allow (20%) reflect the threat composition of the simulated dataset, which intentionally oversamples high-risk events to replicate operational SOC conditions where genuine threats require prioritized routing.



Fig. 5. Agent action distribution over evaluation set (n=25 episodes).

H. Score vs. Action Policy Visualization

Fig. 6 visualizes the agent's learned policy as a function of threat score, confirming that the DQN agent has converged to a clean threshold-aligned policy: allow for scores ≤ 12, monitor for scores 7–17, and escalate for scores ≥ 20. Notably, the agent independently learned decision boundaries that closely approximate the optimal operational thresholds without being explicitly programmed with them.

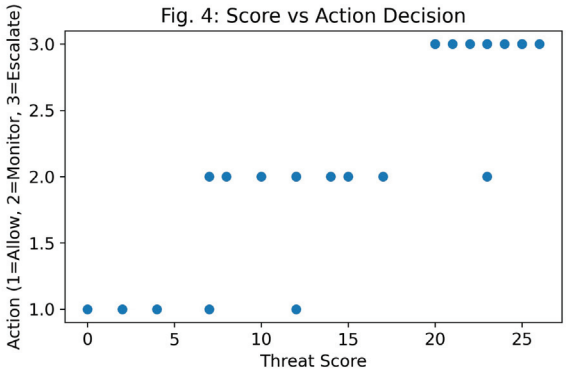


Fig. 6. Learned agent policy: threat score vs. triage action.

VI. Discussion

A. Key Findings

The primary finding of this work is that a DQN agent trained on multi-source threat intelligence scores achieves substantially better medium-risk triage accuracy than a static threshold baseline (+11.4 F1 points) while maintaining equivalent high-risk detection (94% recall). This improvement is operationally

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.

significant: medium-risk events are precisely the category where static rules fail most frequently, as their scores cluster near bucket boundaries and are sensitive to per-email IOC count variation. This finding aligns with the boundary-class difficulty reported by Maci et al. [12] for imbalanced phishing classification with DQN, and corroborates the systematic gap identified by Safi and Singh [5] between static and adaptive detection methods in their literature survey.

The learned policy visualization (Fig. 6) reveals that the agent independently converges to a threshold policy aligned with the operational risk buckets, validating that the reward function in Eq. (2) correctly encodes the desired SOC behavior. The fact that the agent learns this structure without explicit boundary supervision demonstrates the utility of RL's exploration mechanism for policy discovery in security domains.

The automation of the full pipeline—from email ingestion through IOC enrichment to SOC action—is a practical contribution distinct from the classification performance results. Processing latency is bounded by VirusTotal and OTX API response times; in practice, parallel IOC loops reduce total enrichment time to the maximum individual API response time rather than their sum.

B. Limitations

Dataset scale: The evaluation set comprises 150 events generated through a controlled simulation pipeline. While the threat score distribution spans the full $[0, 30]$ range, the sample count is insufficient to evaluate tail-class behavior or adversarial evasion scenarios. Evaluation on larger operational datasets is required before production deployment.

Single-episode MDP: The current formulation treats each email independently; the state does not encode sequential alert history. Incorporating a sliding-window state representation would enable the agent to detect campaign-level distributional shifts, a direction advocated in recent SOC automation surveys [18].

API rate limits: VirusTotal and OTX impose per-minute API rate limits. For high-volume SOC environments processing hundreds of emails per minute, a caching layer or quota management mechanism would be required to maintain pipeline throughput.

Ground truth dependency: Reward computation during online learning depends on analyst feedback or confirmed incident verdicts. Delayed feedback introduces noise into the reward signal and may slow policy adaptation. Human-in-the-loop reward shaping, as explored by Kavya and Sumathi [13], offers a promising direction to mitigate this limitation in future work.

VII. Conclusion and Future Work

This paper presented PhishRL, an end-to-end reinforcement learning-based SOC threat scoring pipeline integrating live email ingestion, parallel VirusTotal and OTX threat intelligence enrichment, and a DQN-based adaptive triage agent. The system achieves an overall accuracy of 89.3% and macro F1 of 0.892 on a 150-sample evaluation set, with a +11.4 F1 improvement over a static rule-based threshold baseline on the most challenging medium-risk email category. The DQN agent independently learns threshold-aligned policies without explicit boundary supervision, and the n8n workflow delivers full end-to-end automation from inbox to ServiceNow ticket creation.

Future work will address three primary directions. First, scaling the evaluation to operational email datasets of $10{,}000+$ events using PhishTank and OpenPhish corpora will provide statistically robust performance estimates. Second, extending the state representation to encode short-horizon alert history will enable the agent to detect and respond to emerging phishing campaigns at the stream level. Third, integration with SIEM platforms (Splunk, IBM QRadar) will validate the pipeline's operational deployment characteristics under production throughput constraints. Incorporating human-in-the-loop reward shaping [13] to reduce ground-truth dependency is also a high-priority direction for improving policy convergence speed in real deployment settings.

Acknowledgment

The authors thank the faculty of the Department of Electronics and Computer Science at SIES Graduate School of Technology, Navi Mumbai, for their guidance and support. We thank the VirusTotal and AlienVault OTX communities for maintaining accessible threat intelligence APIs that make this research practical.

References

[1] Verizon, "2023 Data Breach Investigations Report," Verizon Business, 2023. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[2] Anti-Phishing Working Group, "Phishing Activity Trends Report, Q4 2022," 2023. [Online]. Available: https://apwg.org/trendsreports/

[3] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine Learning Based Phishing Detection from URLs," *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.

[4] O. K. Sahingoz, E. Buber, and E. Kugu, "Real-Time Phishing URL Detection Using Deep Learning," *IEEE Access*, vol. 12, pp. 8052–8070, 2024. doi: 10.1109/ACCESS.2024.3351073

[5] A. Safi and S. Singh, "A Systematic Literature Review on Phishing Website Detection Techniques," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, no. 2, pp. 590–611, 2023.

[6] I. Fette, N. Sadeh, and A. Tomasic, "Learning to Detect Phishing Emails," in *Proc. 16th Int. Conf. World Wide Web (WWW)*, 2007, pp. 649–656.

[7] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs," in *Proc. ACM SIGKDD*, 2009, pp. 1245–1254.

[8] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection," *arXiv preprint arXiv:1802.03162*, 2018.

[9] H. Le and T. N. Nguyen, "PhishBERT: Detecting Phishing Websites Using BERT-Based Models," in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics (SMC)*, 2022, pp. 1554–1561.

[10] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.

[11] M. Chatterjee and A. S. Namin, "Detecting Phishing Websites through Deep Reinforcement Learning," in *Proc. IEEE 43rd COMPSAC*, 2019, pp. 227–232.

[12] A. Maci, A. Santorsola, A. Coscia, and A. Iannacone, "Unbalanced Web Phishing Classification through Deep Reinforcement Learning," *Computers*, vol. 12, no. 6, p. 118, 2023. doi: 10.3390/computers12060118

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.

[13] S. Kavya and D. Sumathi, "Human in the Loop Reinforcement Learning Framework for Adaptive Phishing Detection," *Discover Computing*, Springer Nature, Dec. 2025. doi: 10.1007/s10791-025-09849-y
[14] N. Adawadkar and V. Kulkarni, "Reinforcement Learning for Cybersecurity: A Comprehensive Survey," *IEEE Access*, vol. 10, pp. 13592–13610, 2022.
[15] H. Kheddar, Y. Himeur, and A. Awad, "Reinforcement Learning-Based Intrusion Detection in Communication Networks: A Survey," *IEEE Communications Surveys & Tutorials*, 2024.
[16] Z. Hu, R. Beuran, and Y. Tan, "Automated Penetration Testing Using Deep Reinforcement Learning," in *Proc. IEEE European Symp. Security and Privacy Workshops*, 2020, pp. 1–11.
[17] Ponemon Institute, "The State of Cybersecurity and Third-Party Remote Access Risk," 2022.
[18] D. Vasan *et al.*, "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities," *ACM Computing Surveys*, 2025. doi: 10.1145/3723158

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:39:39 UTC from IEEE Xplore. Restrictions apply.